

The Deterministic Pipeline: Architectural Divergence in Mobile Rendering Engines (Skia vs. Impeller)

Executive Summary

The domain of mobile graphical rendering is currently undergoing a fundamental paradigmatic shift, transitioning from general-purpose, runtime-adaptive libraries to bespoke, ahead-of-time (AOT) compiled rendering runtimes. For over a decade, the industry standard has been defined by Skia, an immediate-mode 2D graphics library that prioritized feature breadth and cross-platform compatibility over low-level architectural specialization. While Skia has successfully powered ecosystems ranging from Google Chrome to Android, its architecture—rooted in the state-machine logic of legacy OpenGL—has increasingly clashed with the stringent latency requirements of modern 120Hz mobile user interfaces.

The emergence of Impeller, Flutter's dedicated rendering engine, represents a rejection of the "general-purpose" graphics model. By inverting the control flow of the rendering pipeline, Impeller prioritizes **predictability** over flexibility. This report provides an exhaustive technical analysis of this transition. We dissect the rendering pipeline at its most granular levels: from the computational geometry of path tessellation to the memory alignment of uniform buffers. The analysis contrasts Skia's reliance on CPU-bound triangulation and Just-In-Time (JIT) shader compilation with Impeller's utilization of GPU-accelerated "Stencil-then-Cover" algorithms and offline shader reflection.

Through a rigorous examination of render passes, memory bandwidth utilization in Tile-Based Deferred Rendering (TBDR) architectures, and command buffer encoding strategies, this report demonstrates that Impeller's architecture eliminates the structural causes of "jank." However, this predictability is achieved through a calculated trade-off: a significant increase in static binary size and runtime memory consumption, necessitated by the storage of pre-compiled pipeline state objects and transient attachment buffers. This document serves as a comprehensive reference for graphics engineers and systems architects seeking to understand the theoretical and practical implications of this divergence in rendering theory.

1. The Theoretical constraints of Modern Mobile Graphics

To fully appreciate the architectural necessity of Impeller, one must first establish the physical and logical constraints of the hardware it inhabits. The mobile System-on-Chip (SoC) imposes a set of limitations vastly different from the desktop GPU environments where legacy APIs like

OpenGL were born.

1.1 The Memory Bandwidth Bottleneck and TBDR

Unlike desktop architectures which often employ Immediate Mode Rendering (IMR)—streaming geometry and pixel operations directly to a frame buffer in VRAM via a high-bandwidth bus—mobile GPUs predominantly utilize **Tile-Based Deferred Rendering (TBDR)**. Architectures from Apple (PowerVR/Apple Silicon), Qualcomm (Adreno), and ARM (Mali) optimize for power efficiency by minimizing external memory access.¹

In a TBDR architecture, the render target is subdivided into a grid of small on-chip tiles (e.g., 16x16 or 32x32 pixels). The rendering process is split into two distinct phases:

1. **Binning (Tiling) Phase:** The GPU processes vertex geometry to determine which primitives overlap with which tiles. This requires valid position data but does not execute fragment shaders.
2. **Rasterization Phase:** The GPU processes each tile individually. Crucially, the frame buffer for the current tile resides in ultra-fast, on-chip static RAM (SRAM). Fragment shading, blending, and depth testing occur within this local memory. Only when the tile is fully complete is the result written out to the main system memory (DRAM).

The implication for rendering engines is profound. Any operation that breaks this "tile-local" execution—such as switching a shader program, binding a new texture, or changing a blend mode—carries a disproportionate cost. If the hardware cannot maintain the state of the current tile during a context switch, it may be forced to resolve the tile to main memory and reload it later, an operation that consumes significant memory bandwidth and battery power. Skia's immediate-mode design, which issues draw calls sequentially as they appear in the code, often struggles to optimize for these hardware-specific pass structures, leading to excessive "Load/Store" traffic. Impeller, by contrast, is designed to be "TBDR-aware," utilizing explicit render passes to manage tile memory lifecycles efficiently.²

1.2 The Latency of State Validation

Legacy graphics APIs like OpenGL ES are designed as state machines. The driver maintains a massive, opaque context containing the current state of the GPU (active textures, bound buffers, current shader program, stencil settings). When the application issues a draw call (e.g., `glDrawArrays`), the driver must validate that the current combination of states is valid. This validation overhead is non-trivial. It involves complex logic to ensure that the vertex attributes matches the shader inputs, that the textures are complete, and that the blend modes are supported. In a complex application drawing thousands of widgets per frame, this CPU-side driver overhead can exceed the time required for the GPU to actually draw the pixels. This phenomenon is often referred to as "driver overhead" or "submit cost." Modern APIs like Metal and Vulkan move this validation to the creation of the **Pipeline State Object (PSO)**. A PSO is an immutable object that bakes all validation into its creation step. If a rendering engine can create PSOs ahead of time, the draw call becomes a simple pointer reference, virtually eliminating driver overhead. Skia's dynamic architecture makes this "baking" difficult; Impeller's static architecture makes it mandatory.⁴

2. Architecture I: The Skia Legacy and the Generalist Trap

Skia has served as the bedrock of Flutter's rendering stack since its inception. To understand why it is being replaced, we must analyze its design philosophy, which prioritizes correctness and ubiquity over the specific performance characteristics of modern mobile GPUs.

2.1 The Ganesh Backend: An Immediate Mode Abstraction

Skia's GPU backend, known as **Ganesh**, operates on an immediate-mode abstraction. When the Flutter framework paints a frame, it constructs a Scene comprising a hierarchy of logical layers. This scene is rasterized by Skia, which translates high-level commands (e.g., `canvas.drawRect`) into low-level GPU API calls (GL or Vulkan).

The core unit of work in Ganesh is the `OpsTask` (formerly `GrOpList`). Skia attempts to batch draw calls by accumulating compatible operations into a single task. For example, if the application draws ten red rectangles in sequence, Ganesh effectively batches these into a single geometry buffer and issues one draw call. While effective for simple scenes, this heuristic breaks down in complex UIs where state changes (changing colors, textures, clips) are frequent. Every break in the batch forces a new draw call and potentially a shader switch.²

2.2 The Combinatorial Explosion of Dynamic Shaders

The most critical limitation of Skia in the context of Flutter is its handling of shaders. Skia is a general-purpose library; it does not know ahead of time what the application will draw. Therefore, it cannot ship with a pre-compiled library of every possible shader. Instead, it employs a sophisticated **shader generation engine**.

When a draw call is issued, Skia inspects the current `SkPaint` object to determine the necessary GPU logic:

- Does the paint use a solid color or a gradient?
- Is anti-aliasing enabled?
- Is there a color filter (e.g., sepia tone)?
- Is there a mask filter (e.g., blur)?
- What is the blending mode (e.g., `SrcOver`, `Multiply`, `Screen`)?

Skia stitches together snippets of GLSL (GL Shading Language) code to form a complete shader program that satisfies these specific conditions. This dynamically generated source code is then handed to the graphics driver for compilation.

2.3 The Anatomy of Shader Compilation Jank

The "Jank" associated with shader compilation is a direct consequence of this JIT model.

1. **Trigger:** The user triggers an animation that introduces a new visual effect (e.g., a hero transition that uses a specific clip and opacity blend for the first time).
2. **Generation:** Skia generates the unique GLSL key for this effect.

3. **Cache Miss:** Skia checks its in-memory cache. If this specific combination has not been seen before, it results in a cache miss.
4. **Compilation Stall:** Skia passes the GLSL to the driver (glCompileShader, glLinkProgram). The driver pauses the application's raster thread while it optimizes and compiles the shader into machine code (ISA).
5. **Frame Drop:** This compilation process is synchronous and unpredictable. On a low-end device, it might take 100ms. Since the frame budget at 60Hz is 16.6ms (and 8.3ms at 120Hz), the frame is inevitably dropped. The user perceives this as a stutter.¹

Attempts to mitigate this via **Shader Warmup** (recording a trace of shaders used during a test run and pre-compiling them) proved fragile. Shaders are binary-incompatible across different GPU vendors and driver versions. A shader cache captured on a Pixel phone is useless on a Samsung phone, making reliable distribution of "warmed up" shaders impossible in the fragmented Android ecosystem.⁹

3. Architecture II: The Impeller Paradigm - Inversion of Control

Impeller was engineered to solve the "Jank" problem by fundamentally inverting the control flow of rendering. Instead of asking "What shader do I need for this draw call?", Impeller asks "How can I map this draw call to the set of shaders I already have?"

3.1 The AOT Philosophy

The guiding principle of Impeller is that **nothing essential should happen at runtime**. All expensive rendering state calculations are shifted to the engine's build time. This is achieved through a bespoke offline compiler pipeline that completely replaces the runtime shader generation of Skia.

3.2 Deep Dive: The impellerc Compiler Infrastructure

The stability of Impeller relies on impellerc, an offline shader compiler that runs on the developer's machine (or Google's build servers). The workflow of impellerc is a sophisticated pipeline of translation and reflection:

1. **Source Input:** Shaders are authored in **GLSL 4.60**. This provides a high-level, standard input format for engine developers.¹⁰
2. **Intermediate Representation (SPIR-V):** The GLSL is compiled into **SPIR-V**, a binary intermediate format standard used by Vulkan. This ensures that the logical behavior of the shader is fixed and validated before the app is ever built.
3. **Reflection & Metadata Extraction:** This is a critical divergence from legacy pipelines. impellerc analyzes the SPIR-V bytecode to extract detailed reflection data. It determines the exact memory layout of all Uniform Buffer Objects (UBOs), the binding slots for textures, and the input attributes.
 - **FlatBuffers/JSON:** This reflection data is serialized into efficient schemas (using

FlatBuffers for runtime speed or JSON for debugging). This metadata allows the runtime engine to know *exactly* how to populate memory for the GPU without needing to query the driver, eliminating a major source of runtime overhead.¹¹

4. Backend Transpilation:

- **iOS (Metal):** The SPIR-V is cross-compiled into **Metal Shading Language (MSL)** and then compiled into a binary metallib.
- **Android (Vulkan):** The SPIR-V is optimized and embedded directly.
- **Android (OpenGL ES Fallback):** For older devices, the SPIR-V is transpiled back to high-performance GLSL ES.¹⁰

3.3 Pipeline State Objects (PSOs)

By having all shaders available as binary blobs at startup, Impeller can construct **Pipeline State Objects (PSOs)** immediately upon application launch. A PSO in Metal/Vulkan is a monolithic object that encapsulates the shader program, blend state, pixel format, and vertex descriptor.

- **Immutability:** Once created, a PSO is immutable.
- **Predictability:** Because PSOs are created upfront, the cost of "switching" pipelines during a frame is reduced to a simple pointer swap. The driver does not need to re-validate the state, because the state was validated when the PSO was created.
- **The Permutation Trade-off:** To support the wide variety of Flutter rendering primitives without dynamic generation, Impeller utilizes a "uvershader" approach less and creates specialized pipelines for common operations, while using specialization constants to handle minor variations. This significantly increases the binary size of the engine, as thousands of pre-compiled shader variants must be shipped with the app. This is the explicit trade-off: **Disk space and startup memory for runtime smoothness.**⁴

4. Computational Geometry: The Tessellation Shift

Perhaps the most mathematically significant divergence between Skia and Impeller is their approach to rendering vector paths. Vector graphics are defined by mathematical curves (Bezier, quadratic, cubic), but GPUs are essentially "triangle processing machines." The process of converting curves to triangles is **Tessellation**.

4.1 Skia's CPU-Bound Approach: Trapezoidalization

Historically, Skia relied on the CPU to perform tessellation.

- **The Algorithms:** Skia uses algorithms (derived from libtess2 and internal developments) to decompose complex paths into non-overlapping trapezoids or triangles. This involves calculating the intersection points of curves, handling self-intersecting paths, and applying winding rules (determining which regions are "inside" the shape) entirely in software.
- **The Bottleneck:** This approach is computationally expensive, often $O(N \log N)$. If an application creates a complex animation—such as a Lottie file where a character's

shape morphs every frame—the CPU must re-calculate the triangulation for the entire path 60 times a second.

- **Bus Saturation:** Once triangulated, the resulting mesh (which can contain thousands of vertices) must be uploaded to the GPU. This consumes PCI-e (or SoC internal bus) bandwidth and creates synchronization points where the CPU must wait for the upload to complete before the GPU can proceed.¹⁴

4.2 Impeller's GPU-Bound Approach: Stencil-then-Cover

Impeller adopts a technique known as **Stencil-then-Cover (StC)**, popularized by NVIDIA's NV_path_rendering extensions. This approach shifts the burden of "figuring out the shape" from the CPU to the GPU's rasterizer and stencil testing units.³

4.2.1 The Algorithm Mechanics

The StC algorithm decouples the definition of the shape from its shading. It executes in two distinct passes for every path:

Phase 1: The Stencil Pass (Analysis)

- **Geometry:** The CPU performs a trivial tessellation. It does *not* calculate intersections or solve curves. It simply generates a "triangle fan" radiating from a central point to the path's edges, or tessellates curves into simple line segments based on a tolerance threshold. This is an $O(N)$ operation and extremely fast.
- **State:** Color writing is disabled. Stencil writing is enabled.
- **Winding Logic:** The GPU draws these triangles. They may overlap messily. However, the **Stencil Buffer** is used to track the "winding number."
 - For every fragment generated by a triangle with a clockwise edge, the stencil value is incremented.
 - For every fragment with a counter-clockwise edge, the value is decremented.
- **Result:** At the end of this pass, the stencil buffer contains a mathematical map of the shape. Any pixel with a non-zero stencil value is "inside" the shape (according to the Non-Zero winding rule). Pixels with a zero value are outside.¹⁵

Phase 2: The Cover Pass (Shading)

- **Geometry:** Impeller draws a simple bounding rectangle (a quad) that covers the extents of the path.
- **State:** Color writing is enabled. The Stencil Test is set to NotEqual(0) (or Odd for Even-Odd rules).
- **Execution:** The GPU rasterizes the quad. For each pixel, it checks the stencil buffer. If the value is non-zero, it executes the fragment shader (applying gradients, textures, or blurs). If zero, it discards the pixel.

4.2.2 Theoretical Implications and Trade-offs

- **CPU Relief:** The CPU load is drastically reduced. It no longer solves geometry; it simply streams vertex data. This frees up the main thread for Dart application logic.
- **Fill Rate Pressure:** The GPU effectively processes the pixels twice (once for stencil,

once for cover). However, modern mobile GPUs have massive fill rate capabilities compared to their CPU single-core performance. Impeller bets on the GPU being the stronger processor.

- **Memory Bandwidth:** The StC approach requires a Stencil Buffer attachment for the render pass. On TBDR architectures, this is efficient *if* the stencil buffer stays in on-chip tile memory. If the render pass is interrupted or too complex, the driver might be forced to spill the stencil buffer to main memory, causing a bandwidth spike. This explains the higher "GL mtrack" usage observed in Impeller apps.¹⁷

5. The Rendering Pipeline: Command Buffers and Batching

Moving data and instructions from the CPU to the GPU is the "bus factor" of rendering performance. Impeller minimizes this overhead through rigorous data management and modern API abstractions.

5.1 The Entity-Component-System of Rendering

Impeller organizes its rendering logic around **Entities**.

- **Entity:** A high-level drawing operation (e.g., a rectangle, a text run, a path). It contains the transformation matrix, the paint state, and the geometry.
- **EntityPass:** A collection of entities that are rendered into a specific render target. The root of the scene is an EntityPass. This hierarchical structure allows Impeller to reason about the scene globally before encoding commands.¹⁹

5.2 Pass Rewriting and Collapsing

One of Impeller's most powerful optimizations is **Pass Collapsing**. In legacy rendering, operations like "saveLayer" (used for opacity, clips, or offscreen rendering) would unconditionally create a new texture and a new render pass. This is expensive on TBDR hardware.

Impeller analyzes the EntityPass tree to find redundancies:

- **Clip Collapsing:** If a layer is created solely to apply a clip, Impeller checks if the clip can be applied directly to the parent pass using the stencil buffer. If so, the layer is dissolved, and the clip is encoded as a state change in the parent pass.
- **Merge Compatibility:** If two consecutive passes write to the same target and share compatible states, they are merged.
- **Result:** This significantly reduces the number of "Render Pass" boundaries, which are the most expensive operations on mobile GPUs due to the required memory flush/reload.¹⁹

5.3 Multi-Threaded Command Encoding

Skia (Ganesh) is generally single-threaded in its submission logic. Impeller leverages the

command buffer architecture of Metal and Vulkan to support concurrency.

- **Parallel Encoding:** While the main raster thread orchestrates the frame, Impeller can offload the encoding of distinct EntityPass trees to worker threads. Each thread records commands into a secondary command buffer.
- **Submission:** These buffers are then joined and submitted to the GPU queue in a single batch. This ensures that the CPU can saturate the GPU's command queue without being bottlenecked by a single core's serial processing speed.⁵

5.4 Uniform Buffers and Reflection-Based Binding

In Skia, setting a uniform (e.g., the opacity of a layer) might involve finding the uniform location by string name or updating a slot in a generic array. Impeller uses the reflection data from `impellerc` to map uniforms directly to memory.

- **Host-Visible Memory:** Impeller allocates large **Uniform Buffer Objects (UBOs)** in host-visible memory.
- **Direct Memcpy:** Because `impellerc` has generated C++ structs that match the shader's memory layout (accounting for padding and alignment rules like `std140`), Impeller can simply memcpy the data structure into the buffer.
- **Offset Binding:** The draw call binds this large buffer at a specific dynamic offset. This "bind-less" style (or minimal binding) approach removes the overhead of individual `glUniform` calls, which heavily tax the driver.²

6. Memory Architecture and Bandwidth

The transition to Impeller involves a conscious decision to trade memory for performance. This section analyzes the specific mechanisms of this trade-off.

6.1 Transient Attachments and Tile Memory

To support the Stencil-then-Cover algorithm without exploding memory bandwidth, Impeller relies heavily on **Transient Attachments**.

- **Mechanism:** In Metal (using `.memoryless` storage mode) and Vulkan (using `VK_IMAGE_USAGE_TRANSIENT_ATTACHMENT_BIT` and lazy allocation), Impeller creates stencil and depth buffers that *only* exist in the GPU's on-chip tile memory.
- **The Bandwidth Saving:** These buffers are never written to system DRAM. The GPU uses them during the processing of a tile and discards them immediately after the tile is done. This allows Impeller to use heavy multi-pass stencil algorithms with virtually zero memory bandwidth cost.¹⁸

6.2 Ring Buffering and Resource Lifecycle

To ensure the CPU never waits for the GPU (blocking), Impeller employs a **Ring Buffer** strategy for dynamic resources (vertices, uniforms).

- **Triple Buffering:** Impeller typically maintains 3 generations of resources. While Frame N is being displayed, Frame N+1 is being rendered by the GPU, and Frame N+2 is being

encoded by the CPU.

- **Implication:** This effectively triples the memory footprint for volatile buffers compared to a synchronous, single-buffered approach. This is a primary contributor to the increased RAM usage observed in Impeller applications. However, it ensures that the "CPU idle time" is maximized, as the CPU rarely blocks waiting for a buffer to become available.¹⁸

6.3 The "GL mtrack" Phenomenon

Developers often observe that "GL mtrack" (graphics memory tracking) is higher and monotonically increasing in Impeller compared to Skia.

- **Allocator Behavior:** Skia tends to be aggressive about freeing resources to keep its footprint low. Impeller employs a **Heaped Allocator** strategy. It allocates blocks of memory (slabs) and sub-allocates from them. It holds onto these slabs even when empty, anticipating future frames.
- **Stability:** This prevents memory fragmentation and the overhead of constant malloc/free calls to the OS, which can cause jitter. The memory usage appears high but is stable, whereas Skia's usage would "sawtooth," potentially triggering garbage collection or OS-level memory pressure events at inopportune times.¹⁷

7. Performance Analysis: Case Studies

The theoretical architectural differences manifest in concrete performance characteristics.

7.1 Blur and Image Processing

Blur rendering provides a distinct example of the architectural divergence.

- **Skia:** Historically used a multi-pass downscaling approach on the CPU or GPU. This often triggered runtime shader compilation for specific kernel sizes (sigma values), leading to the "first-run jank" on blurred backdrops.
- **Impeller:** Implements a standard Gaussian blur using a **separable kernel** (Pass 1: Horizontal Blur, Pass 2: Vertical Blur).
- **Optimization:** Impeller leverages the **Tile Memory** optimizations described in Section 6.1. By configuring the render pass to LoadAction: Clear and StoreAction: Store only when necessary, and keeping intermediate blur results in tile memory where possible, recent updates have shown Impeller reducing blur rendering time by nearly 50% compared to Skia. The throughput matches specialized implementations while maintaining the flexibility of the general pipeline.³

7.2 Lottie and Complex Vector Animation

In scenarios involving complex vector animations (e.g., Lottie files), the difference is stark.

- **Skia:** The CPU usage spikes linearly with the complexity of the path (number of vertices/curves) due to re-tessellation. At high complexity, the main thread saturates, dropping frames.

- **Impeller:** The CPU usage remains low and constant, as it simply streams the vertex data. The load shifts to the GPU. Since the GPU is highly parallel, it can process the "Stencil-then-Cover" operations for thousands of vertices without stalling the UI thread. Benchmarks indicate that Impeller can maintain 120fps in scenes where Skia struggles to maintain 60fps.²⁴

8. Conclusion: The Deterministic Future

The transition from Skia to Impeller represents the maturation of the Flutter framework from a "cross-platform UI toolkit" to a "high-performance graphical runtime." It acknowledges that the "general-purpose" abstraction of Skia, while incredibly powerful and theoretically correct, contains inherent unpredictability—specifically JIT shaders and CPU-bound geometry—that is incompatible with the industry's move toward 120Hz displays and rich, motion-heavy user interfaces.

8.1 Summary of Key Divergences

Feature	Skia (Ganesh)	Impeller	Architectural Implication
Pipeline Generation	Runtime (JIT)	Offline (AOT)	Eliminates runtime compilation jank; Increases binary size.
Tessellation	CPU (Trapezoidal)	GPU (Stencil-then-Cover)	Offloads CPU; Uses more GPU memory bandwidth (stencil).
State Management	State Machine (GL)	Immutable PSOs	Eliminates driver validation overhead.
Memory Strategy	Aggressive Release	Ring Buffering & Pooling	Higher, stable RAM usage; Consistent frame times.
Uniforms	Slot/String lookup	Reflection (FlatBuffers)	Type-safe, direct memory mapping.

8.2 Future Outlook

Impeller's architecture aligns Flutter directly with the design principles of modern graphics APIs. By treating the GPU as a command-driven co-processor rather than a state machine, Impeller not only solves the immediate problem of jank but also lays the foundation for future capabilities. The explicit control over command buffers paves the way for the upcoming **Flutter GPU API**, which will allow developers to write custom compute shaders and highly optimized rendering logic that integrates seamlessly with the standard widget tree. In this deterministic future, the rendering engine ceases to be a black box and becomes a

transparent, predictable conduit between the developer's intent and the user's display.

Works cited

1. How Impeller Is Transforming Flutter UI Rendering in 2026 - DEV Community, accessed December 11, 2025, <https://dev.to/eira-wexford/how-impeller-is-transforming-flutter-ui-rendering-in-2026-3dpg>
2. Understanding Flutter's New Rendering Engine: Impeller - Vibe Studio, accessed December 11, 2025, <https://vibe-studio.ai/insights/understanding-flutter-s-new-rendering-engine-impeller>
3. What's new in Flutter 3.22. WebAssembly, Graphics rendering... | by Kevin Chisholm, accessed December 11, 2025, <https://blog.flutter.dev/whats-new-in-flutter-3-22-fbde6c164fe3>
4. Understanding Impeller — Flutter's Modern Rendering Engine | by Ammar Yasser - Medium, accessed December 11, 2025, <https://medium.com/@ImAmmarYasser/understanding-impeller-flutters-modern-rendering-engine-13e04ed5fde4>
5. Impeller · flutter/flutter Wiki · GitHub, accessed December 11, 2025, <https://github.com/flutter/flutter/wiki/Impeller/fad846d4c206ae8cab6285cc182a5d9b1af5717b>
6. Flutter's Architecture (Part-IV): Skia Vs. Impeller Engines | by Os & 1s - Medium, accessed December 11, 2025, <https://medium.com/@Os.and.1s/flutter-part-iv-skia-vs-impeller-graphics-engine-s-33327e408ba8>
7. Flutter Performance Tips: Reducing Shader Compilation Jank - Vibe Studio, accessed December 11, 2025, <https://vibe-studio.ai/insights/flutter-performance-tips-reducing-shader-compilation-jank>
8. Shader compilation jank in Flutter | by Chamith Athukorala - Medium, accessed December 11, 2025, <https://medium.com/@chamithmathukorala/shader-compilation-jank-in-flutter-8cc9d92aa8b3>
9. Shader compilation jank | Flutter - Your Name, accessed December 11, 2025, https://liudonghua123.github.io/flutter_website/perf/shader/
10. impeller - external/github.com/flutter/engine - Git at Google, accessed December 11, 2025, https://chromium.googlesource.com/external/github.com/flutter/engine/+refs/heads/master_archived/impeller
11. impeller/tools/impeller.gni - external/github.com/flutter/engine - Git at Google, accessed December 11, 2025, <https://chromium.googlesource.com/external/github.com/flutter/engine/+refs/heads/flutter-3.18-candidate.4/impeller/tools/impeller.gni>
12. Writing and using fragment shaders - Flutter documentation, accessed

- December 11, 2025, <https://docs.flutter.dev/ui/design/graphics/fragment-shaders>
13. Flutter's next-generation graphics renderer, Impeller, accessed December 11, 2025, <https://segmentfault.com/a/1190000042416588/en>
 14. [Impeller] 🌂 Path drawing performance needs significant improvement. #143077 - GitHub, accessed December 11, 2025, <https://github.com/flutter/flutter/issues/143077>
 15. [Impeller] Why did Impeller opt for "triangulation then draw" over "stencil then cover"? · Issue #123671 · flutter/flutter - GitHub, accessed December 11, 2025, <https://github.com/flutter/flutter/issues/123671>
 16. GPU-accelerated Path Rendering - NVIDIA, accessed December 11, 2025, <https://developer.download.nvidia.com/devzone/devcenter/gamegraphics/files/opengl/gpupathrender.pdf>
 17. Impeller slow to release GPU memory compared to SKIA. Crashes more often. - Reddit, accessed December 11, 2025, https://www.reddit.com/r/flutterhelp/comments/1ok7c7u/impeller_slow_to_release_gpu_memory_compared_to/
 18. engine/impeller/docs/read_frame_captures.md at main - GitHub, accessed December 11, 2025, https://github.com/flutter/engine/blob/main/impeller/docs/read_frame_captures.md
 19. impeller/entity/entity_pass.cc - external/github.com/flutter/engine - Git at Google, accessed December 11, 2025, https://chromium.googlesource.com/external/github.com/flutter/engine/+/refs/heads/flutter-3.17-candidate.6/impeller/entity/entity_pass.cc
 20. impeller::Entity Class Reference - Flutter, accessed December 11, 2025, https://api.flutter.dev/impeller/classimpeller_1_1_entity.html
 21. Flutter 3.24.0 release notes, accessed December 11, 2025, <https://docs.flutter.dev/release/release-notes/release-notes-3.24.0>
 22. [Impeller] Parallelize generation of render pass hierarchies. · Issue #106519 - GitHub, accessed December 11, 2025, <https://github.com/flutter/flutter/issues/106519>
 23. What's new in Flutter 3.19. Revolutionizing App Development with... | by Kevin Chisholm, accessed December 11, 2025, <https://blog.flutter.dev/whats-new-in-flutter-3-19-58b1aae242d2>
 24. Explore the Flutter New Update: Key Features in Flutter 3.22 - Ingenious Minds Lab, accessed December 11, 2025, <https://ingeniousmindslab.com/blogs/explore-the-flutter-new-update-3-22/>
 25. Skia vs Impeller: A Performance Comparison | by Raiden Lpf - Medium, accessed December 11, 2025, <https://medium.com/@raidен.lpf666/skia-vs-impeller-a-performance-comparison-e1c7dfd9e861>